

Word Sense Disambiguation with WordNet and Graph Centrality

The word **bank** means something different in *"He sat on the bank of the river"* and *"The bank approved the loan"*. Picking the right meaning from context is called **Word Sense Disambiguation (WSD)**.

A neat way to do WSD without any training data is to treat **WordNet as a graph** and let **network centrality measures** vote for the most "well-connected" sense of each word. This guide walks through the idea and demonstrates implementation using NLTK and NetworkX with the following centrality measures:

- Degree centrality
- Betweenness centrality
- Closeness centrality
- PageRank
- Hubs and Authorities (HITS)
- Key Player Problem (KPP)

1. The core idea

1. A word has several **senses** (WordNet *synsets*), e.g. `bank.n.01` (river bank) and `depository_financial_institution.n.01` (money bank).
2. Words in the same sentence usually talk about the same topic, so **the correct senses of neighbouring words should be semantically close** in WordNet.
3. Build a graph whose nodes are candidate senses, and connect senses through WordNet's hypernym ("is-a") links.
4. The correct senses end up **densely connected** to each other. Wrong senses remain on the periphery or isolated.
5. Compute a centrality score for each candidate sense and **pick the highest-scoring sense per word**.

2. Setup

Install the necessary libraries via pip:

```
pip install nltk networkx
```

Import dependencies and download the WordNet database:

```
import itertools
```

```
import networkx as nx
import nltk
from nltk.corpus import wordnet as wn

nltk.download("wordnet")
nltk.download("omw-1.4")
```

We use nouns only, so all candidate senses live in one unified hypernym taxonomy and can be linked to each other.

3. Step 1: Get candidate senses

```
words = ["bank", "river", "water"]
candidates = {w: wn.synsets(w, pos=wn.NOUN) for w in words}
# 'bank' -> 10 senses, 'river' -> 1, 'water' -> 6
```

4. Step 2: Build the semantic graph

Why not link every sense to the root?

The naive approach is to add the full hypernym path (entity -> ... -> sense) for every candidate. Do not do this. Every sense then shares high-level generic ancestors such as entity or abstraction, making the entire graph artificially connected so that abstract root nodes dominate every centrality metric.

Better: connect senses of different words through short paths

Following the classic graph-based WSD approach (Navigli and Lapata):

- For every pair of senses belonging to **different words**, find their **Lowest Common Subsumer (LCS)**, the most specific shared ancestor.
- If the two senses are within a maximum threshold distance (e.g., 6 edges), add the two chains sense -> ... -> LCS to the graph.
- Senses with no short connection to any other word's candidate senses remain **isolated**.

```
MAX_PATH_LEN = 6

def path_up(sense, ancestor):
    """Hypernym chain sense -> ... -> ancestor."""
    for p in sense.hypernym_paths():
        if ancestor in p:
            return p[p.index(ancestor):][::-1]

G, D = nx.Graph(), nx.DiGraph()          # undirected and directed versions
```

```

for s in itertools.chain(*candidates.values()):
    G.add_node(s); D.add_node(s)          # keep every candidate, even if isolated

for w1, w2 in itertools.combinations(words, 2):
    for s1 in candidates[w1]:
        for s2 in candidates[w2]:
            dist = s1.shortest_path_distance(s2)
            if dist is None or dist > MAX_PATH_LEN:
                continue
            for lcs in s1.lowest_common_hyponyms(s2):
                for s in (s1, s2):
                    chain = path_up(s, lcs)
                    for child, parent in zip(chain, chain[1:]):
                        G.add_edge(child, parent)
                        D.add_edge(child, parent)      # direction: sense -> ancestor

```

We maintain two separate graphs:

- **Undirected G:** Used for degree, betweenness, closeness, PageRank, and KPP.
- **Directed D (sense → ancestor):** Used for HITS, which fundamentally relies on directed link asymmetry.

5. Step 3: Score candidate senses with centrality

Each measure asks a distinct topological question about node importance:

5.1 Degree centrality

Number of neighbours normalized by $n - 1$. Senses connected to many other candidate senses have high local popularity.

```
deg = nx.degree_centrality(G)
```

5.2 Betweenness centrality

Fraction of all shortest paths passing through a node. High values highlight **bridges** linking distinct topological clusters.

```
btw = nx.betweenness_centrality(G)
```

5.3 Closeness centrality

Reciprocal of the average shortest path distance to all reachable nodes. High values mean the concept is **geometrically close to everything**.

```
clo = nx.closeness centrality(G)
```

5.4 PageRank

A node is important if other important nodes link to it. Modeled via random walks with damping (default = 0.85).

```
pr = nx.pagerank(G)
```

5.5 Hubs and Authorities (HITS)

In a directed network, nodes receive dual complementary scores:

- **Authority:** Pointed to by many prominent hubs.
- **Hub:** Points outward to many prominent authorities.

In our sense → ancestor formulation, shared ancestors serve as natural authorities, while candidate senses linking into them serve as hubs. Ranking senses by **hub score** isolates the best candidates. Below is a robust power-iteration implementation:

```
def hits(D, iterations=50):
    hub = {n: 1.0 for n in D}
    for _ in range(iterations):
        auth = {n: sum(hub[u] for u in D.predecessors(n)) for n in D}
        hub = {n: sum(auth[v] for v in D.successors(n)) for n in D}
        norm = sum(x * x for x in hub.values()) ** 0.5 or 1.0
        hub = {n: x / norm for n, x in hub.items()} # normalize each round
    return hub, auth
```

Note: nx.hits fails on edge-empty graphs, and its randomized solver can yield slight floating-point variations. The deterministic manual implementation resolves both issues.

5.6 Key Player Problem (KPP)

Borgatti's **Key Player Problem** determines the most vital nodes in a network under two formulations:

KPP-Neg (Fragmentation): Which node, when removed, fragments the network into the maximum number of disconnected components?

$$F = 1 - \frac{\sum (s_k * (s_k - 1))}{(n * (n - 1))} \quad s_k = \text{size of } k\text{-th component}$$

```
def kpp_neg(G):
    n, scores = G.number_of_nodes(), {}
    for node in G:
        H = G.copy()
        H.remove_node(node)
        comps = [len(c) for c in nx.connected_components(H)]
        scores[node] = 1 - sum(s * (s - 1) for s in comps) / (n * (n - 1))
    return scores
```

KPP-Pos (Reach): Which node (or set of nodes) is positioned to reach all other graph nodes with minimal path length?

$$\text{reach}(S) = \sum_{v \text{ not in } S} 1 / d(S, v) \quad d(S, v) = \text{distance from nearest member of } S$$

```
def kpp_pos(G, S):
    dist = nx.multi_source_dijkstra_path_length(G, S)
    return sum(1 / d for v, d in dist.items() if v not in S and d > 0)
```

For individual WSD scoring, each node is evaluated as a single-node set: $\{n: \text{kpp_pos}(G, [n]) \text{ for } n \text{ in } G\}$.

To find the optimal cohesive set of k nodes, a greedy search selects the candidate providing maximum incremental reach:

```
def greedy_kpp_pos(G, k):
    chosen = []
    for _ in range(k):
        best = max((n for n in G if n not in chosen),
                   key=lambda n: kpp_pos(G, chosen + [n]))
        chosen.append(best)
    return chosen
```

Key players identified in this manner reflect concepts that unify the underlying theme (e.g. `body_of_water.n.01` for river-related context).

6. Step 4: Choose the sense

For each word, select the candidate with the highest centrality score, incorporating two safeguards:

- **Ties:** Isolated senses score 0. Break ties by picking the first listed synset, taking advantage of WordNet's frequency ordering (Most Frequent Sense heuristic).
- **Floating-point noise:** Round scores prior to comparison (e.g. `round(score, 6)`) to avoid arbitrary tie-breaks driven by floating point precision artifacts.

```
def disambiguate(words, scores, candidates):
    result = {}
    for w in words:
        best = max(enumerate(candidates[w]),
                    key=lambda t: (round(scores[t[1]], 6), -t[0]))
        result[w] = best[1]
    return result

hubs, _ = hits(D)
all_scores = {"Degree": deg, "Betweenness": btw, "Closeness": clo,
              "PageRank": pr, "HITS (hub)": hubs,
              "KPP-Neg": kpp_neg(G),
              "KPP-Pos": {n: kpp_pos(G, [n]) for n in G}}

for name, scores in all_scores.items():
    print(name, {w: s.name() for w, s in disambiguate(words, scores,
candidates).items()})
```

7. Results

Hand-labelled senses across three target sentences (6 evaluation words total):

Sentence	Expected Synsets
He sat on the bank of the river and watched the water	bank.n.01 (sloping land), river.n.01
She moved the mouse and typed on the keyboard	mouse.n.04 (device), keyboard.n.01
The bank approved the loan and charged interest	depository_financial_institution.n.01, loan.n.01

Evaluation performance across centrality metrics:

Centrality Measure	Correct Senses (out of 6)
Degree	6 / 6
KPP-Neg	6 / 6
Closeness	5 / 6

Centrality Measure	Correct Senses (out of 6)
KPP-Pos	5 / 6
Betweenness	4 / 6
PageRank	4 / 6
HITS (hub)	4 / 6

Disclaimer: With only six annotated target words, this is an illustrative comparison rather than a formal statistical benchmark. The core value lies in understanding distinct topological behavior.

8. Observations

- **Connected senses dominate:** In the river sentence, the sloping land sense of bank is selected effortlessly because financial senses lack short taxonomic paths to river or water.
- **Measures can disagree:** In the finance context, Closeness and KPP-Pos selected bank.n.06 ("funds held by a gambling house") due to its proximity to general money-related nodes, whereas Degree, PageRank, and KPP-Neg correctly pinpointed depository_financial_institution.n.01.
- **Global measures favor the graph center:** Closeness, KPP-Pos, and PageRank reward nodes positioned centrally in the overall component. An incorrect sense that is globally central can outscore a correct sense located in a smaller, tightly clustered branch.
- **Bridge-based measures are sensitive to topology:** Betweenness and KPP-Neg award points only to articulation points and bridges. On small graphs with few alternative paths, score granularity is coarse, leading to frequent ties.
- **WordNet hierarchy limits accuracy:** Related terms like bank (institution) and loan (transaction) belong to disjoint top-level taxonomy branches. Without non-hypernym relations (meronymy, glosses, domain labels), short hypernym paths cannot bridge them.

9. Tips and limitations

- **Sentence Length Matters:** More content words provide richer topological evidence. With only two words, the graph is sparse and fragile.
- **Tune MAX_PATH_LEN:** A threshold too low isolates relevant concepts; too high causes spurious connections through broad ancestors. Values between 4 and 6 are recommended.
- **Noun-Only Restriction:** Verbs, adjectives, and adverbs do not share hypernym hierarchies with nouns. Integrating other parts of speech requires cross-POS semantic links or gloss overlaps.

- **Compare Against Baseline:** The Most Frequent Sense (MFS) baseline (`wn.synsets(word)[0]`) is very strong in WordNet. Always verify that graph algorithms surpass this baseline.
- **Computational Overhead:** KPP-Neg simulates graph removal per node, and greedy KPP-Pos evaluates every node at each step. Both scale well for small sentence subgraphs but require optimization for document-scale graphs.

10. Summary

Step	Procedure
1	Extract content words (nouns) from the input context
2	Retrieve all candidate synsets for each word using <code>wn.synsets(word, pos=wn.NOUN)</code>
3	Connect cross-word sense pairs via short paths through their Lowest Common Subsumer (LCS) to form undirected and directed graphs
4	Compute centrality scores (Degree, Betweenness, Closeness, PageRank, HITS hub, KPP-Neg, KPP-Pos)
5	Select the maximum-scoring sense per word, breaking ties using the Most Frequent Sense heuristic
6	Compare outputs against ground truth and evaluate relative strengths across graph metrics